- On the master MongoDB, enter the following command:

```
mongo
rs.initiate()
rs.add("<hostname/IP of slave>:<port number on which Mongo
is running, by default it runs on 27017>")
```

- On the slave MongoDB, enter the following command:

```
mongo
rs.initiate()
```

Note: If there are an even number of slaves, it's advisable to add an arbiter to the setup. An arbiter is needed as a secondary cannot vote for itself; so, sometimes a situation arises when the votes are tied.

- Start the arbiter. Install MongoDB. Create the arbiter data directory *mkdir -p /data/arb* (this can be anything you want). Start the arbiter service:

```
mongod --port 3000 --dbpath /data/arb/ --replset
<replication set name used>
```

- Enter the following command on the master MongoDB to add the arbiter to the setup:

```
mongo

rs.addArb("<hostname/IP of arbiter>:3000")
```

- Use *rs.status()* to display the status of the replication set
- Another way of checking for the members in replica is to use the URL *http://<machine-name>:<port>/_replSet*

Note: The port number is 1000 ahead of the MongoDB port; for example, if the Mongo port is 27017, here the port will be 28017.

Sharding

The config server processes are *mongod* instances that store the cluster's metadata. You designate a *mongod* as a config server using the *–configsvr* option. Each config server stores a complete copy of the cluster's metadata. In production deployments, you must deploy exactly three config server instances, each running on different servers to ensure good uptime and data safety. In test environments, you can run all three instances on a single server.

Steps to add sharding to the current setup

- Install MongoDB as described earlier.
- Create data directories for each of the three config server instances. By default, a config server stores its data files in the */data/configdb* directory. You can choose a different location. To create a data directory, run a command similar to the following:

```
mkdir -p /data/configdb
```

- Start the config server instances as follows:

```
mongod --configsvr --dbpath /data/configdb --port 22019(you
can use any port)
```

- Start the *mongos* instance. The *mongos* instances are lightweight and do not require data directories. You can run a *mongos* instance on a system that runs other cluster components, such as on an application server or a server running a *mongod* process.

```
mongos --configdb <config server hostnames/IP>
```

Add shards to the cluster

- On the primary Mongo, issue the following command:

```
mongo --host <hostname of machine running mongos> --port
<port mongos listens on>
```

- On the *mongos* shell got by the above command, enter the following command:

```
sh.addShard( "<replication set name>/<Mongo
server1>:<port>,<Mongo server2>:<port>,<Mongo
server3>:<port>" )
```

Enable sharding for the database:

```
sh.enableSharding("<Name of the database you need to
shard")
```

Now check the sharding status:

```
mongo
use admin
db.printShardingstatus()
```

You can enable sharding for a collection as follows:

- Determine what you will use for the shard key.
- If the collection already contains data, you must create an index on the shard key using *ensureIndex()*. If the collection is empty, then MongoDB will create the index as part of the *sh.shardCollection()* step.
- Enable sharding for a collection by issuing the *sh.shardCollection()* method in the Mongo shell. The method uses the following syntax:

```
sh.shardCollection("<database>.<collection>", shard-key-
pattern selected above)
```